

**РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
ФАКУЛЬТЕТ ФИЗИКО-МАТЕМАТИЧЕСКИХ И
ЕСТЕСТВЕННЫХ НАУК**

Лабораторная работа №.9

Понятие подпрограммы. Отладчик GDB.

Левищева Анастасия Петровна, НКАбд-02-25

Содержание

Список иллюстраций.....	3
1. Цель работы.....	4
2. Задание.....	5
3. Теоретическое введение.....	6
4. Выполнение лабораторной работы.....	15
5. Выводы.....	31
Список литературы.....	32

Список иллюстраций

Рис.4.1.1.Каталог.....	15
Рис.4.1.2.Исполняемый файл(1).....	15
Рис.4.1.3.Текст программы(1).....	16
Рис.4.1.4.Исполняемый файл(2).....	17
Рис.4.2.1.Файл lab09-2.asm.....	17
Рис.4.2.2.Исполняемый файл(3).....	17
Рис.4.2.3.Загрузка в отладчик(1).....	18
Рис.4.2.4.Проверка.....	18
Рис.4.2.5.Брейкпоинт.....	18
Рис.4.2.6.Дисассемблированный код.....	19
Рис.4.2.7. Intel'овский синтаксис.....	19
Рис.4.2.8.Псевдографика.....	20
Рис.4.2.1.1.команда i b.....	21
Рис.4.2.1.2.Точка останова.....	21
Рис.4.2.1.3.Все точки останова.....	21
Рис.4.2.2.1.Значение переменной(1).....	22
Рис.4.2.2.2.Значение переменной(2).....	22
Рис.4.2.2.3.Изменение(1).....	23
Рис.4.2.2.4.Изменение(2).....	23
Рис.4.2.2.5.Значение регистра edx.....	23
Рис.4.2.2.6.Изменение регистра.....	23
Рис.4.2.2.5.Завершение и выход.....	24
Рис.4.2.3.1.Копия.....	24
Рис.4.2.3.2.Исполняемый файл(4).....	25
Рис.4.2.3.3.Загрузка в отладчик(2).....	25
Рис.4.2.3.3.Загрузка в отладчик(2).....	25
Рис.4.2.3.5.Кол-во аргументов.....	26
Рис.4.2.3.6.Остальные позиции стека.....	26
Рис.1.Файл var4-1.asm.....	27
Рис.2.Текст программы(2).....	27
Рис.3.Исполняемый файл(5).....	28
Рис.4.Файл var4-2.asm.....	28
Рис.5.Исполняемый файл(3).....	28
Рис.5.Исполняемый файл(4).....	29

1. Цель работы

Приобретение навыков написания программ с использованием подпрограмм.
Знакомство с методами отладки при помощи GDB и его основными возможностями.

2. Задание

1. Преобразуйте программу из лабораторной работы №8 (Задание №1 для самостоятельной работы), реализовав вычисление значения функции $f(x)$ как подпрограмму.
2. В листинге 9.3 приведена программа вычисления выражения $(3 + 2) * 4 + 5$. При запуске данная программа дает неверный результат. Проверьте это. С помощью отладчика GDB, анализируя изменения значений регистров, определите ошибку и исправьте ее.

Листинг 9.3. Программа вычисления выражения $(3 + 2) * 4 + 5$

```
%include      'in_out.asm'

SECTION .data
div:  DB 'Результат: ',0

SECTION .text
GLOBAL _start
_start:

; ---- Вычисление выражения (3+2)*4+5
    mov ebx,3
    mov eax,2
    add ebx,eax
    mov ecx,4
    mul ecx
    add ebx,5
    mov edi,ebx

; ---- Вывод результата на экран
    mov eax,div
    call sprint
    mov eax,edi
    call iprintLF

    call quit
```

3. Теоретическое введение

3.1. Понятие об отладке

Отладка — это процесс поиска и исправления ошибок в программе. В общем случае его можно разделить на четыре этапа:

- обнаружение ошибки;
- поиск её местонахождения;
- определение причины ошибки;
- исправление ошибки.

Можно выделить следующие типы ошибок:

- синтаксические ошибки — обнаруживаются во время трансляции исходного кода и вызваны нарушением ожидаемой формы или структуры языка;
- семантические ошибки — являются логическими и приводят к тому, что программа запускается, отработывает, но не даёт желаемого результата;
- ошибки в процессе выполнения — не обнаруживаются при трансляции и вызывают прерывание выполнения программы (например, это ошибки, связанные с переполнением или делением на ноль).

Второй этап — поиск местонахождения ошибки. Некоторые ошибки обнаружить довольно трудно. Лучший способ найти место в программе, где находится ошибка, это разбить программу на части и произвести их отладку отдельно друг от друга.

Третий этап — выяснение причины ошибки. После определения местонахождения ошибки обычно проще определить причину неправильной работы программы.

Последний этап — исправление ошибки. После этого при повторном запуске программы, может обнаружиться следующая ошибка, и процесс отладки начнётся заново.

3.2. Методы отладки

Наиболее часто применяют следующие методы отладки:

- создание точек контроля значений на входе и выходе участка программы (например, вывод промежуточных значений на экран — так называемые диагностические сообщения);
- использование специальных программ-отладчиков.

Отладчики позволяют управлять ходом выполнения программы, контролировать и изменять данные. Это помогает быстрее найти место ошибки в программе и ускорить её исправление. Наиболее популярные способы работы с отладчиком — это использование точек останова и выполнение программы по шагам.

Пошаговое выполнение — это выполнение программы с остановкой после каждой строчки, чтобы программист мог проверить значения переменных и выполнить другие действия.

Точки останова — это специально отмеченные места в программе, в которых программа-отладчик приостанавливает выполнение программы и ждёт команд. Наиболее популярные виды точек останова:

- Breakpoint — точка останова (остановка происходит, когда выполнение доходит до определённой строки, адреса или процедуры, отмеченной программистом);
- Watchpoint — точка просмотра (выполнение программы приостанавливается, если программа обратилась к определённой переменной: либо считала её значение, либо изменила его).

Точки останова устанавливаются в отладчике на время сеанса работы с кодом программы, т.е. они сохраняются до выхода из программы-отладчика или до смены отлаживаемой программы.

3.3. Основные возможности отладчика GDB

GDB (GNU Debugger — отладчик проекта GNU) [1] работает на многих UNIX-подобных системах и умеет производить отладку многих языков программирования. GDB предлагает обширные средства для слежения и контроля за выполнением компьютерных программ. Отладчик не содержит собственного графического пользовательского интерфейса и использует стандартный текстовый интерфейс консоли. Однако для GDB существует несколько сторонних графических надстроек, а кроме того, некоторые интегрированные среды разработки используют его в качестве базовой подсистемы отладки.

Отладчик GDB (как и любой другой отладчик) позволяет увидеть, что происходит «внутри» программы в момент её выполнения или что делает программа в момент сбоя.

GDB может выполнять следующие действия:

- начать выполнение программы, задав всё, что может повлиять на её поведение;
- остановить программу при указанных условиях;
- исследовать, что случилось, когда программа остановилась;
- изменить программу так, чтобы можно было поэкспериментировать с устранением эффектов одной ошибки и продолжить выявление других.

3.3.1. Запуск отладчика GDB; выполнение программы; выход

Синтаксис команды для запуска отладчика имеет следующий вид:

```
gdb [опции] [имя_файла | ID процесса]
```

После запуска `gdb` выводит текстовое сообщение — так называемое «nice GDB logo». В следующей строке появляется приглашение (`gdb`) для ввода команд.

Далее приведён список некоторых команд GDB.

Команда `run` (сокращённо `r`) — запускает отлаживаемую программу в оболочке GDB.

Если точки останова не были установлены, то программа выполняется и выводятся сообщения:

```
(gdb) run
Starting program: test
Program exited normally.
(gdb)
```

Если точки останова были заданы, то отладчик останавливается на соответствующей команде и выдаёт номер точки останова, адрес и дополнительную информацию — текущую строку, имя процедуры, и др.

Команда `kill` (сокращённо `k`) прекращает отладку программы, после чего следует вопрос о прекращении процесса отладки:

```
Kill the program being debugged? (y or n) y
```

Если в ответ введено `y` (то есть «да»), отладка программы прекращается. Командой `run` её можно начать заново, при этом все точки останова (breakpoints), точки просмотра (watchpoints) и точки отлова (catchpoints) сохраняются.

Для выхода из отладчика используется команда `quit` (или сокращённо `q`):

```
(gdb) q
```

3.3.2. Дизассемблирование программы

Если есть файл с исходным текстом программы, а в исполняемый файл включена информация о номерах строк исходного кода, то программу можно отлаживать, работая в отладчике непосредственно с её исходным текстом. Чтобы программу можно было отлаживать на уровне строк исходного кода, она должна быть откомпилирована с ключом `-g`.

Посмотреть дизассемблированный код программы можно с помощью команды `disassemble` :

```
(gdb) disassemble _start
```

Существует два режима отображения синтаксиса машинных команд: режим Intel, используемый в том числе в NASM, и режим АТТ (значительно

отличающийся внешне). По умолчанию в дизассемблере GDB принят режим АТТ. Переключиться на отображение команд с привычным Intel'овским синтаксисом можно, введя команду `set disassembly-flavor intel`

3.3.3. Точки останова

Установить точку останова можно командой `break` (кратко `b`). Типичный аргумент этой команды — место установки. Его можно задать как имя метки или как адрес. Чтобы не было путаницы с номерами, перед адресом ставится «звёздочка»:

```
(gdb) break *<адрес>
(gdb) b <метка>
```

Информацию о всех установленных точках останова можно вывести командой `info` (кратко `i`):

```
(gdb) info breakpoints
(gdb) i b
```

Для того чтобы сделать неактивной какую-нибудь ненужную точку останова, можно воспользоваться командой `disable`:

```
disable breakpoint <номер точки останова>
```

Обратно точка останова активируется командой `enable`:

```
enable breakpoint <номер точки останова>
```

Если же точка останова в дальнейшем больше не нужна, она может быть удалена с помощью команды `delete`:

```
(gdb) delete breakpoint <номер точки останова>
```

Ввод этой команды без аргумента удалит все точки останова.

Информацию о командах этого раздела можно получить, введя

```
help breakpoints
```

3.3.4. Пошаговая отладка

Для продолжения остановленной программы используется команда `continue` (`c`) (`gdb`) с [аргумент]. Выполнение программы будет происходить до следующей точки останова. В качестве аргумента может использоваться

целое число N , которое указывает отладчику проигнорировать $N - 1$ точку останова (выполнение остановится на N -й точке).

Команда `stepi` (кратко `sI`) позволяет выполнять программу по шагам, т.е. данная команда выполняет ровно одну инструкцию:

```
(gdb) si [аргумент]
```

При указании в качестве аргумента целого числа N отладчик выполнит команду `step` N раз при условии, что не будет точек останова или выполнение программы не прервётся по другим причинам.

Команда `nexthi` (или `ni`) аналогична `stepi`, но вызов процедуры (функции) трактуется отладчиком как одна инструкция:

```
(gdb) ni [аргумент]
```

Информацию о командах этого раздела можно получить, введя

```
(gdb) help running
```

3.3.5. Работа с данными программы в GDB

Как уже упоминалось, отладчик может показывать содержимое ячеек памяти и регистров, а при необходимости позволяет вручную изменять значения регистров и переменных.

Посмотреть содержимое регистров можно с помощью команды `info registers` (или `i r`):

```
(gdb) info registers
```

Для отображения содержимого памяти можно использовать команду `x/NFU`, выдаёт содержимое ячейки памяти по указанному адресу. `NFU` задает формат, в котором выводятся данные (см. Таблицу 9.1).

Таблица 9.1. Формат отображения данных команды `x`.

	Значение	Описание
N	Десятичное целое число	Счётчик повторений. Определяет, сколько ячеек памяти отобразить (считая в единицах), по умолчанию 1.
F	Формат отображения	
	<code>s</code>	строка, оканчивающаяся нулём
	<code>i</code>	машинная инструкция
	<code>x</code>	шестнадцатеричное число
	<code>a</code>	адрес
U	Размер отображаемых ячеек памяти	
	<code>b</code>	байт
	<code>h</code>	полуслово, 2 байта
	<code>w</code>	машинное слово, 4 байта (значение по умолчанию)
	<code>g</code>	длинное слово, 8 байт

Например, `x/4uh 0x63450`— это запрос на вывод четырёх полуслов (`h`) из памяти в формате беззнаковых десятичных целых (`u`), начиная с адреса `0x63450`.

Чтобы посмотреть значения регистров используется команда `print /F` (сокращенно `p`). Перед именем регистра обязательно ставится префикс `$`. Например, команда `p/x $ecx` выводит значение регистра в шестнадцатеричном формате.

Изменить значение для регистра или ячейки памяти можно с помощью команды `set`, задав ей в качестве аргумента имя регистра или адрес. При этом перед именем регистра ставится префикс `$`, а перед адресом нужно указать в фигурных скобках тип данных (размер сохраняемого значения; в качестве типа данных можно использовать типы языка Си).

Справку о любой команде `gdb` можно получить, введя

```
(gdb) help [имя_команды]
```

3.3.6. Понятие подпрограммы

Подпрограмма — это, как правило, функционально законченный участок кода, который можно многократно вызывать из разных мест программы. В отличие от простых переходов из подпрограмм существует возврат на команду, следующую за вызовом.

Если в программе встречается одинаковый участок кода, его можно оформить в виде подпрограммы, а во всех нужных местах поставить её вызов. При этом подпрограмма будет содержаться в коде в одном экземпляре, что позволит уменьшить размер кода всей программы.

3.3.6.1. Инструкция call и инструкция ret

Для вызова подпрограммы из основной программы используется инструкция `call`, которая заносит адрес следующей инструкции в стек и загружает в регистр `еір` адрес соответствующей подпрограммы, осуществляя таким образом переход. Затем начинается выполнение подпрограммы, которая, в свою очередь, также может содержать подпрограммы.

Подпрограмма завершается инструкцией `ret`, которая извлекает из стека адрес, занесённый туда соответствующей инструкцией `call`, и заносит его в `еір`. После этого выполнение основной программы возобновится с инструкции, следующей за инструкцией `call`.

Подпрограмма может вызываться как из внешнего файла, так и быть частью основной программы.

Основные моменты выполнения подпрограммы иллюстрируются на рис. 9.1.



Рис. 9.1. Основные моменты выполнения подпрограммы

Важно помнить, что если в подпрограмме занести что-то в стек и не извлечь, то на вершине стека окажется не адрес возврата и это приведёт к ошибке выхода из подпрограммы. Кроме того, надо помнить, что подпрограмма без команды возврата не вернётся в точку вызова, а будет выполнять следующий за подпрограммой код, как будто он является её продолжением.

4. Выполнение лабораторной работы

4.1. Реализация подпрограмм в NASM

Создадим каталог для выполнения лабораторной работы № 9, перейдем в него и создадим файл lab09-1.asm:

```
aplevishcheva@debian:~$ mkdir ~/work/arch-pc/lab09
aplevishcheva@debian:~$ cd ~/work/arch-pc/lab09
aplevishcheva@debian:~/work/arch-pc/lab09$ touch lab09-1.asm
aplevishcheva@debian:~/work/arch-pc/lab09$ ls
lab09-1.asm
```

Рис.4.1.1.Каталог

В качестве примера рассмотрим программу вычисления арифметического выражения $f(x) = 2x + 7$ с помощью подпрограммы `_calcul`.

Внимательно изучим текст программы (Листинг 9.1)

Первые строки программы отвечают за вывод сообщения на экран (`call sprint`), чтение данных, введенных с клавиатуры (`call sread`) и преобразования введенных данных из символьного вида в численный (`call atoi`).

После следующей инструкции `call _calcul`, которая передает управление подпрограмме `_calcul`, будут выполнены инструкции подпрограммы.

Инструкция `ret` является последней в подпрограмме и ее исполнение приводит к возвращению в основную программу к инструкции, следующей за инструкцией `call`, которая вызвала данную подпрограмму.

Последние строки программы реализуют вывод сообщения (`call sprint`), результата вычисления (`call iprintLF`) и завершение программы (`call quit`).

Введем в файл lab09-1.asm текст программы из листинга 9.1. Создадим исполняемый файл и проверим его работу:

```
aplevishcheva@debian:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
aplevishcheva@debian:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
aplevishcheva@debian:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 5
2x+7 = 17
```

Рис.4.1.2.Исполняемый файл(1)

Программа работает корректно.

Изменим текст программы, добавив подпрограмму `_subcalcul` в подпрограмму `_calcul`, для вычисления выражения $f(g(x))$, где x вводится с клавиатуры, $f(x) = 2x + 7$, $g(x) = 3x - 1$:

```
/home/aplevis~9/lab09-1.asm  [----] 12 L: [ 1+42 43/ 57] *(477 / 628b) 0010 0x00A [*][X]
%include 'in_out.asm'

section .data
    msg:      db 'Введите x: ', 0
    result:   db 'f(g(x)) = ', 0

section .bss
    x:        RESB 80
    res:      RESB 80
    subres:   RESB 80

section .text
global _start

_start:
    mov     eax, msg
    call    sprint

    mov     ecx, x
    mov     edx, 80
    call    sread

    mov     eax, x
    call    atoi

    call    _calcul

    mov     eax, result
    call    sprint
    mov     eax, [res]
    call    iprintLF

    call    quit

_calcul:
    call    _subcalcul

    mov     ebx, 2
    mul     ebx
    add     eax, 7
    mov     [res], eax

    ret

_subcalcul:
    mov     ebx, 3
    mul     ebx
    sub     eax, 1

    mov     [subres], eax

    ret

1Помощь 2Сохран 3Блок 4Замена 5Копия 6Пер~ить 7Поиск 8Удалить 9МенюМС 10Выход
```

Рис.4.1.3.Текст программы(1)

x передается в подпрограмму `_calcul` из нее в подпрограмму `_subcalcul`, где вычисляется выражение $g(x)$, результат возвращается в `_calcul` и вычисляется выражение $f(g(x))$. Результат возвращается в основную программу для вывода результата на экран.

Создадим исполняемый файл и проверим его работу:

```
aplevishcheva@debian:~/work/arch-pc/lab09$ nasm -f elf lab09-1.asm
aplevishcheva@debian:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-1 lab09-1.o
aplevishcheva@debian:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 2
f(g(x)) = 17
aplevishcheva@debian:~/work/arch-pc/lab09$ ./lab09-1
Введите x: 1
f(g(x)) = 11
aplevishcheva@debian:~/work/arch-pc/lab09$
```

Рис.4.1.4.Исполняемый файл(2)

Программа работает корректно.

4.2. Отладка программ с помощью GDB

Создадим файл `lab09-2.asm` с текстом программы из Листинга 9.2. (Программа печати сообщения Hello world!):

```
aplevishcheva@debian:~/work/arch-pc/lab09$ touch lab09-2.asm
aplevishcheva@debian:~/work/arch-pc/lab09$ ls
in_out.asm lab09-1 lab09-1.asm lab09-1.o lab09-2.asm
aplevishcheva@debian:~/work/arch-pc/lab09$
```

Рис.4.2.1.Файл lab09-2.asm

Получим исполняемый файл:

```
aplevishcheva@debian:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-2.lst lab09-2.asm
aplevishcheva@debian:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-2 lab09-2.o
aplevishcheva@debian:~/work/arch-pc/lab09$ ./lab09-2
Hello, world!
aplevishcheva@debian:~/work/arch-pc/lab09$
```

Рис.4.2.2.Исполняемый файл(3)

Для работы с GDB в исполняемый файл необходимо добавить отладочную информацию, для этого трансляцию программ необходимо проводить с ключом `'-g'`.

Загрузим исполняемый файл в отладчик gdb:

```
aplevishcheva@debian:~/work/arch-pc/lab09$ gdb lab09-2
GNU gdb (Debian 16.3-1) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-2...
(gdb) █
```

Рис.4.2.3.Загрузка в отладчик(1)

Проверим работу программы, запустив ее в оболочке GDB с помощью команды run (сокращённо r):

```
(gdb) run
Starting program: /home/aplevishcheva/work/arch-pc/lab09/lab09-2
Hello, world!
[Inferior 1 (process 3276) exited normally]
(gdb) █
```

Рис.4.2.4.Проверка

Для более подробного анализа программы установим брейкпоинт на метку `_start`, с которой начинается выполнение любой ассемблерной программы, и запустим её:

```
(gdb) break _start
Breakpoint 1 at 0x8049000: file lab09-2.asm, line 14.
(gdb) run
Starting program: /home/aplevishcheva/work/arch-pc/lab09/lab09-2

Breakpoint 1, _start () at lab09-2.asm:14
14      mov     eax, 4
(gdb) █
```

Рис.4.2.5.Брейкпоинт

Посмотрим дисассемблированный код программы с помощью команды `disassemble` начиная с метки `_start`:

```

(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:    mov     $0x4,%eax
    0x08049005 <+5>:    mov     $0x1,%ebx
    0x0804900a <+10>:   mov     $0x804a000,%ecx
    0x0804900f <+15>:   mov     $0x8,%edx
    0x08049014 <+20>:   int     $0x80
    0x08049016 <+22>:   mov     $0x4,%eax
    0x0804901b <+27>:   mov     $0x1,%ebx
    0x08049020 <+32>:   mov     $0x804a008,%ecx
    0x08049025 <+37>:   mov     $0x7,%edx
    0x0804902a <+42>:   int     $0x80
    0x0804902c <+44>:   mov     $0x1,%eax
    0x08049031 <+49>:   mov     $0x0,%ebx
    0x08049036 <+54>:   int     $0x80
End of assembler dump.
(gdb) █

```

Рис.4.2.6. Дisasсемблированный код

Переключимся на отображение команд с Intel'овским синтаксисом, введя команду `set disassembly-flavor intel`:

```

(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:    mov     eax,0x4
    0x08049005 <+5>:    mov     ebx,0x1
    0x0804900a <+10>:   mov     ecx,0x804a000
    0x0804900f <+15>:   mov     edx,0x8
    0x08049014 <+20>:   int     0x80
    0x08049016 <+22>:   mov     eax,0x4
    0x0804901b <+27>:   mov     ebx,0x1
    0x08049020 <+32>:   mov     ecx,0x804a008
    0x08049025 <+37>:   mov     edx,0x7
    0x0804902a <+42>:   int     0x80
    0x0804902c <+44>:   mov     eax,0x1
    0x08049031 <+49>:   mov     ebx,0x0
    0x08049036 <+54>:   int     0x80
End of assembler dump.
(gdb) █

```

Рис.4.2.7. Intel'овский синтаксис

Различия отображения синтаксиса машинных команд в режимах АТТ и Intel:

1. Порядок операндов:
 - АТТ: Операнды записываются в порядке источник, назначение (source, destination).
 - Intel: Операнды записываются в порядке назначение, источник (destination, source).
2. Синтаксис регистров:
 - АТТ: Регистры обозначаются с префиксом `%`. Например, регистр `eax` будет записываться как `%eax`.
 - Intel: Регистры записываются без префикса. Например, `eax`.

3. Использование размеров данных:

- АТТ: Размер данных указывается суффиксом после имени команды. Например, `movb` для байта, `movw` для слова и `movl` для двойного слова.
- Intel: Размер данных указывается неявно по контексту команды или с помощью отдельных команд, таких как `MOV BYTE PTR`, `MOV WORD PTR`, и т.д.

4. Адресация:

- АТТ: использует различные синтаксисы для указания адресации, например, `(%eax)` для обращения к адресу, на который указывает регистр `eax`.
- Intel: использует другой синтаксис, например, `eax` для обращения к адресу в регистре.

5. Комментарии:

- АТТ: Комментарии начинаются с символа `#`.
- Intel: Комментарии начинаются с символа `;`.

Включим режим псевдографики, используя команды `layout asm` и `layout regs`, для более удобного анализа программы:

The screenshot shows a debugger window with two main sections. The top section, titled "Register group: general", displays the current values of the general-purpose registers. The bottom section shows a list of assembly instructions with their addresses and operands.

Register	Value
eax	0x0
ecx	0x0
edx	0x0
ebx	0x0
esp	0xffffd0d0
ebp	0x0
esi	0x0
edi	0x0
eip	0x8049000
eflags	0x202
cs	0x23
ss	0x2b
ds	0x2b

Address	Instruction
0x8049000	<code>mov eax, 0x4</code>
0x8049005	<code>mov ebx, 0x1</code>
0x804900a	<code>mov ecx, 0x804a000</code>
0x804900f	<code>mov edx, 0x8</code>
0x8049014	<code>int 0x80</code>
0x8049016	<code>mov eax, 0x4</code>
0x804901b	<code>mov ebx, 0x1</code>
0x8049020	<code>mov ecx, 0x804a008</code>
0x8049025	<code>mov edx, 0x7</code>
0x804902a	<code>int 0x80</code>
0x804902c	<code>mov eax, 0x1</code>
0x8049031	<code>mov ebx, 0x0</code>
0x8049036	<code>int 0x80</code>
0x8049038	<code>add BYTE PTR [eax], al</code>

native process 3290 (asm) In: start L14 PC: 0x8049000
(gdb) layout regs
(gdb)

Рис.4.2.8.Псевдографика

В этом режиме есть три окна:

- В верхней части видны названия регистров и их текущие значения;
- В средней части виден результат дисассемблирования программы;
- Нижняя часть доступна для ввода команд.

4.2.1. Добавление точек останова

Установить точку останова можно командой `break` (кратко `b`). Типичный аргумент этой команды — место установки. Его можно задать или как номер строки программы (имеет смысл, если есть исходный файл, а программа компилировалась с информацией об отладке), или как имя метки, или как адрес. Чтобы не было путаницы с номерами, перед адресом ставится «звёздочка».

На предыдущих шагах была установлена точка останова по имени метки (`_start`). Проверим это с помощью команды `info breakpoints` (кратко `i b`):

```
(gdb) i b
Num      Type           Disp Enb Address      What
1        breakpoint     keep y   0x08049000   lab09-2.asm:14
(gdb)
```

Рис.4.2.1.1.команда `i b`

Установим еще одну точку останова по адресу инструкции. Определим адрес предпоследней инструкции (`mov ebx,0x0`) и установим точку останова:

```
b+ 0x8049031 <_start+49>   mov     ebx,0x0
    0x8049036 <_start+54>   int     0x80

exec No process (asm) In:                               L??  PC: ??
(gdb) layout regs
(gdb) break *0x8049031
Breakpoint 2 at 0x8049031: file lab09-2.asm, line 27.
(gdb)
```

Рис.4.2.1.2.Точка останова

Посмотрим информацию о всех установленных точках останова:

```
(gdb) i b
Num      Type           Disp Enb Address      What
1        breakpoint     keep y   0x08049000   lab09-2.asm:14
2        breakpoint     keep y   0x08049031   lab09-2.asm:27
```

Рис.4.2.1.3.Все точки останова

4.2.2. Работа с данными программы в GDB

Отладчик может показывать содержимое ячеек памяти и регистров, а при необходимости позволяет вручную изменять значения регистров и переменных.

Выполним 5 инструкций с помощью команды `stepi` (или `si`) и проследим за изменением значений регистров. Изменяются значения следующих регистров:

1. Общие регистры.
2. Регистры сегмента.
3. Флаги.
4. Указатели.

Посмотреть содержимое регистров также можно с помощью команды `info registers` (или `i r`).

Для отображения содержимого памяти можно использовать команду `x`, которая выдаёт содержимое ячейки памяти по указанному адресу. Формат, в котором выводятся данные, можно задать после имени команды через косую черту: `x/NFU <адрес>`.

С помощью команды `x & <имя переменной>` также можно посмотреть содержимое переменной.

Посмотрим значение переменной `msg1` по имени:

```
(gdb) x/1sb &msg1
0x804a000 <msg1>:      "Hello, "
(gdb) █
```

Рис.4.2.2.1.Значение переменной(1)

Посмотрим значение переменной `msg2` по адресу:

```
(gdb) x/1sb 0x804a008
0x804a008 <msg2>:      "world!\n"
```

Рис.4.2.2.2.Значение переменной(2)

Адрес переменной можно определить по дизассемблированной инструкции. Посмотрим инструкцию `mov esx, msg2`, которая записывает в регистр `esx` адрес переменной `msg2`.

Изменить значение для регистра или ячейки памяти можно с помощью команды `set`, задав ей в качестве аргумента имя регистра или адрес. При этом перед именем регистра ставится префикс `$`, а перед адресом нужно указать в фигурных скобках тип данных (размер сохраняемого значения; в качестве типа данных можно использовать типы языка Си).

Изменим первый символ переменной `msg1` (рис. 9.5):

```
(gdb) set {char}msg1='h'
```

Рис.4.2.2.3.Изменение(1)

Заменяем любой символ во второй переменной `msg2`:

```
(gdb) set {char}msg2 = 'w'
```

Рис.4.2.2.4.Изменение(2)

Чтобы посмотреть значения регистров используется команда `print /F <val>` (перед именем регистра обязательно ставится префикс `$`) (рис. 9.6).

Выведем в различных форматах (в шестнадцатеричном формате, в двоичном формате и в символьном виде) значение регистра `edx`:

```
(gdb) p/x $edx
$1 = 0x0
(gdb) p/t $edx
$2 = 0
(gdb) p/c $edx
$3 = 0 '\000'
```

Рис.4.2.2.5.Значение регистра `edx`

С помощью команды `set` изменим значение регистра `ebx`:

```
(gdb) set $ebx = '2'
(gdb) p/s $ebx
$4 = 50
(gdb) set $ebx = 2
(gdb) p/s $ebx
$5 = 2
```

Рис.4.2.2.6.Изменение регистра

Разница вывода команд p/s \$ebx:

- Выражение '2' — это символьная константа. В ASCII код символа '2' равен 0x32 = 50, поэтому после
set \$ebx = '2'
значение регистра ebx становится 50.
- Выражение 2 — обычное целое, поэтому после
set \$ebx = 2
ebx = 2.

Завершим выполнение программы с помощью команды continue (сокращенно c) или stepi (сокращенно si) и выйдем из GDB с помощью команды quit (сокращенно q):

```
(gdb) c
The program is not being run.
(gdb) quit
aplevishcheva@debian:~/work/arch-pc/lab09$
```

Рис.4.2.2.5. Завершение и выход

4.2.3. Обработка аргументов командной строки в GDB

Скопируем файл lab8-2.asm, созданный при выполнении лабораторной работы №8, с программой выводящей на экран аргументы командной строки (Листинг 8.2) в файл с именем lab09-3.asm:

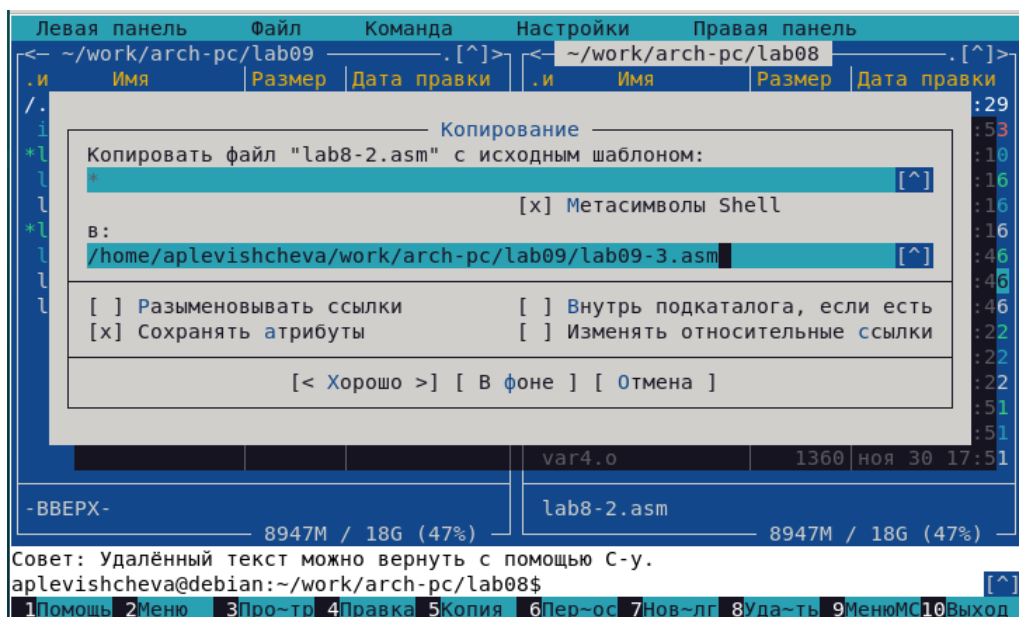


Рис.4.2.3.1. Копия

Создадим исполняемый файл:

```
aplevishcheva@debian:~/work/arch-pc/lab09$ nasm -f elf -g -l lab09-3.lst lab09-3.asm
aplevishcheva@debian:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab09-3 lab09-3.o
aplevishcheva@debian:~/work/arch-pc/lab09$
```

Рис.4.2.3.2.Исполняемый файл(4)

Для загрузки в gdb программы с аргументами необходимо использовать ключ `--args`.

Загрузим исполняемый файл в отладчик, указав аргументы:

```
aplevishcheva@debian:~/work/arch-pc/lab09$ gdb --args lab09-3 аргумент1 аргумент 2 'аргумент 3'
GNU gdb (Debian 16.3-1) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab09-3...
(gdb) █
```

Рис.4.2.3.3.Загрузка в отладчик(2)

Как отмечалось в предыдущей лабораторной работе, при запуске программы аргументы командной строки загружаются в стек. Исследуем расположение аргументов командной строки в стеке после запуска программы с помощью gdb.

Для начала установим точку останова перед первой инструкцией в программе и запустим ее:

```
.....
(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab09-3.asm, line 8.
(gdb) run
Starting program: /home/aplevishcheva/work/arch-pc/lab09/lab09-3 аргумент1 аргумент 2 аргумент\ 3

Breakpoint 1, _start () at lab09-3.asm:8
8      pop    ecx
(gdb)
```

Рис.4.2.3.4.Запуск с помощью gdb

Адрес вершины стека храниться в регистре `esp` и по этому адресу располагается число равное количеству аргументов командной строки (включая имя программы):

```
(gdb) x/x $esp
0xffffd080:      0x05
(gdb)
```

Рис.4.2.3.5. Кол-во аргументов

Как видно, число аргументов равно 5 – это имя программы lab09-3 и непосредственно аргументы: аргумент1, аргумент, 2 и 'аргумент 3'.

Посмотрим остальные позиции стека – по адресу [esp+4] располагается адрес в памяти, где находится имя программы, по адресу [esp+8] храниться адрес первого аргумента, по адресу [esp+12] – второго и т.д.:

```
(gdb) x/s *(void**)( $esp + 4)
0xffffd239:  "/home/aplevishcheva/work/arch-pc/lab09/lab09-3"
(gdb) x/s *(void**)( $esp + 8)
0xffffd268:  "аргумент1"
(gdb) x/s *(void**)( $esp + 12)
0xffffd27a:  "аргумент"
(gdb) x/s *(void**)( $esp + 16)
0xffffd28b:  "2"
(gdb) x/s *(void**)( $esp + 20)
0xffffd28d:  "аргумент 3"
(gdb) x/s *(void**)( $esp + 24)
0x0:      <error: Cannot access memory at address 0x0>
(gdb)
```

Рис.4.2.3.6. Остальные позиции стека

Шаг изменения адреса равен 4 ([esp+4], [esp+8], [esp+12] и т.д.), потому что на этой системе указатель/слово занимает 4 байта.

Задания для самостоятельной работы.

Задание 1(стр.5)

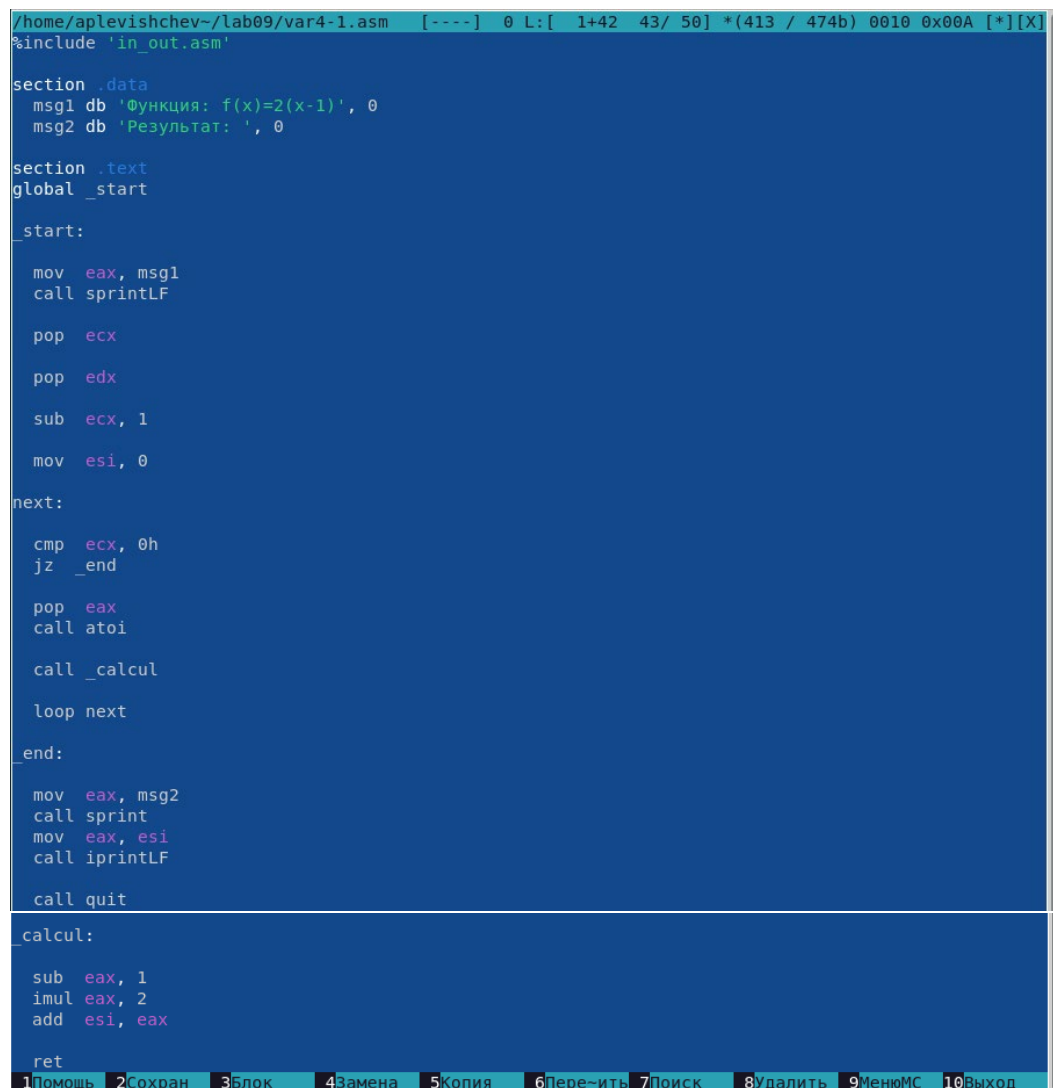
В ходе лабораторной работы №6 был вычислен мой вариант – 4.

Создадим файл var4-1.asm в каталоге lab09:

```
|aplevishcheva@debian:~/work/arch-pc/lab09$ touch var4-1.asm
```

Рис.1.Файл var4-1.asm

Перепишем текст программы для вычисления суммы функций с использованием `_calcul`:



```
/home/aplevishchev~/lab09/var4-1.asm  [----]  0 L:[ 1+42 43/ 50] *(413 / 474b) 0010 0x00A [*][X]
%include 'in_out.asm'

section .data
    msg1 db 'Функция: f(x)=2(x-1)', 0
    msg2 db 'Результат: ', 0

section .text
global _start

_start:

    mov     eax, msg1
    call    sprintLF

    pop     ecx
    pop     edx

    sub     ecx, 1
    mov     esi, 0

next:

    cmp     ecx, 0h
    jz      _end

    pop     eax
    call    atoi

    call    _calcul

    loop    next

_end:

    mov     eax, msg2
    call    sprint
    mov     eax, esi
    call    iprintLF

    call    quit

_calcul:

    sub     eax, 1
    imul    eax, 2
    add     esi, eax

    ret

1Помощь 2Сохран 3Блок 4Замена 5Копия 6Пере~ить 7Поиск 8Удалить 9МенюМС 10Выход
```

Рис.2.Текст программы(2)

Создадим исполняемый файл и проверим его работу:

```
aplevishcheva@debian:~/work/arch-pc/lab09$ nasm -f elf -g var4-1.asm
aplevishcheva@debian:~/work/arch-pc/lab09$ ld -m elf_i386 -o var4-1 var4-1.o
aplevishcheva@debian:~/work/arch-pc/lab09$ ./var4-1 2 1 3
Функция: f(x)=2(x-1)
Результат: 6
aplevishcheva@debian:~/work/arch-pc/lab09$ █
```

Рис.3.Исполняемый файл(5)

Проверим правильность выполнения программы аналитически. Результаты совпадают.

Задание 2(стр.5)

Создадим файл var4-2.asm в каталоге lab09:

```
aplevishcheva@debian:~/work/arch-pc/lab09$ touch var4-2.asm
aplevishcheva@debian:~/work/arch-pc/lab09$ ls
in_out.asm  lab09-1.o  lab09-2.lst  lab09-3.asm  var4-1      var4-2.asm
lab09-1     lab09-2    lab09-2.o   lab09-3.lst  var4-1.asm
lab09-1.asm lab09-2.asm lab09-3     lab09-3.o   var4-1.o
aplevishcheva@debian:~/work/arch-pc/lab09$ █
```

Рис.4.Файл var4-2.asm

Перепишем текст Листинга 9.3, создадим исполняемый файл и посмотрим результат до исправления ошибки:

```
aplevishcheva@debian:~/work/arch-pc/lab09$ nasm -f elf -g var4-2.asm
aplevishcheva@debian:~/work/arch-pc/lab09$ ld -m elf_i386 -o var4-2 var4-2.o
aplevishcheva@debian:~/work/arch-pc/lab09$ ./var4-2
Результат: 10
aplevishcheva@debian:~/work/arch-pc/lab09$ █
```

Рис.5.Исполняемый файл(3)

Результат выполнения программы – 10, а должен быть 25.

Анализируя изменения значений регистров с помощью отладчика GDB, определим ошибку:

В листинге сумма (3+2) хранится в ebx, а перед mul в eax лежит 2, поэтому выполняется 2*4, а не 5*4.

Исправим ошибку и создадим исполняемый файл:

```
aplevishcheva@debian:~/work/arch-pc/lab09$ nasm -f elf -g var4-2.asm
aplevishcheva@debian:~/work/arch-pc/lab09$ ld -m elf_i386 -o var4-2 var4-2.o
aplevishcheva@debian:~/work/arch-pc/lab09$ ./var4-2
Результат: 25
aplevishcheva@debian:~/work/arch-pc/lab09$ █
```

Рис.5.Исполняемый файл(4)

Проверим правильность выполнения программы аналитически. Результаты совпадают.

5. Выводы

В ходе выполнения лабораторной работы были успешно освоены навыки написания программ с использованием подпрограмм. Изучены и применены на практике методы отладки при помощи GDB. Также были получены базовые знания о понятии подпрограммы.

Список литературы

1. GDB: The GNU Project Debugger. — URL: <https://www.gnu.org/software/gdb/>.
2. GNU Bash Manual. — 2016. — URL: <https://www.gnu.org/software/bash/manual/>.
3. Midnight Commander Development Center. — 2021. — URL: <https://midnight-commander.org/>.
4. NASM Assembly Language Tutorials. — 2021. — URL: <https://asmtutor.com/>.
5. Newham C. Learning the bash Shell: Unix Shell Programming. — O'Reilly Media, 2005. — 354 с. — (In a Nutshell). — ISBN 0596009658. — URL: <http://www.amazon.com/Learningbash-Shell-Programming-Nutshell/dp/0596009658>.
6. Robbins A. Bash Pocket Reference. — O'Reilly Media, 2016. — 156 с. — ISBN 978-1491941591.
7. The NASM documentation. — 2021. — URL: <https://www.nasm.us/docs.php>.
8. Zarrelli G. Mastering Bash. — Packt Publishing, 2017. — 502 с. — ISBN 9781784396879.
9. Колдаев В. Д., Лупин С. А. Архитектура ЭВМ. — М. : Форум, 2018.
10. Куляс О. Л., Никитин К. А. Курс программирования на ASSEMBLER. — М. : Солон-Пресс, 2017.
11. Новожилов О. П. Архитектура ЭВМ и систем. — М. : Юрайт, 2016.
12. Расширенный ассемблер: NASM. — 2021. — URL: <https://www.opennet.ru/docs/RUS/nasm/>.
13. Робачевский А., Немнюгин С., Стесик О. Операционная система UNIX. — 2-е изд. — БХВПетербург, 2010. — 656 с. — ISBN 978-5-94157-538-1.
14. Столяров А. Программирование на языке ассемблера NASM для ОС Unix. — 2-е изд. — М. : МАКС Пресс, 2011. — URL: http://www.stolyarov.info/books/asm_unix.
15. Таненбаум Э. Архитектура компьютера. — 6-е изд. — СПб. : Питер, 2013. — 874 с. — (Классика Computer Science).
16. Таненбаум Э., Бос Х. Современные операционные системы. — 4-е изд. — СПб. : Питер, 2015. — 1120 с. — (Классика Computer Science).